Sistema distribuido de reducción adaptativa de ruido con filtrado FIR y espectral sobre microcontroladores

Nicolás Florez Jiménez, Tamara Vanessa Cajiao Molina, Santiago Robles Obando, Santiago Boza Quiros
Escuela de Ingeniería en Computadores, Instituto Tecnológico de Costa Rica, Cartago, Costa Rica
CE 1110 Análisis de Señales Mixtas, II Semestre 2026

***Resumen*—This paper presents a three-node wired system that acquires a noisy signal, characterizes its noise and reduces it in quasi real time with two independent strategies. An RP2040 acquisition node samples the input at 16 kHz with DMA, removes the DC component, applies a Hann window and a 256-point FFT for diagnosis, and streams 256-sample blocks to an ESP32 through SPI frames with a sequence identifier and a CRC-8 checksum. The ESP32 runs two independent processing paths: a 75-tap Kaiser FIR low-pass filter with linear phase and a block spectral method based on a 1024-point radix-2 FFT, 50 % overlap-add and an adaptive relative threshold that preserves conjugate symmetry. The output is played through an 8-bit DAC followed by a second-order Sallen-Key reconstruction filter; a host computer only stores data, tunes parameters and relays playback. A Python model with a direct DFT and FIR and IIR candidates guided the design. Quality metrics are computed on the ESP32 over controlled signals with a clean reference, and its outputs agree with Python by more than 99.99 %; microphone captures validate the physical chain by listening and spectra. For noise outside the useful band the FIR filter improved the SNR by about 60 dB and the spectral method by about 19 dB, with worst-case block times of 8.8 ms and 18.9 ms against budgets of 16 ms and 32 ms. When noise overlaps the useful band neither method improves the SNR without removing useful energy, which quantifies the trade-off between noise reduction and distortion.**

***Index Terms*—noise reduction, FIR filter, spectral filtering, FFT, embedded systems, distributed acquisition**

I. Introducción

Toda señal adquirida desde un sensor llega acompañada de interferencias: tonos de equipos cercanos, ruido de banda ancha del propio circuito o perturbaciones impulsivas. Reducirlas en el mismo sistema embebido que adquiere la señal, sin depender de una computadora, exige elegir con cuidado la frecuencia de muestreo, el tamaño de bloque, el formato numérico y el método de filtrado, porque memoria, tiempo de cálculo y ancho de banda del enlace son limitados [1].

Este trabajo diseña, implementa y evalúa un sistema de tres nodos cableados: un nodo de adquisición y análisis en una Raspberry Pi Pico y dos rutas de procesamiento independientes en una ESP32, una temporal (FIR) y otra espectral (FFT/IFFT) con cálculo de métricas. Ambos métodos se diseñaron primero en un modelo de Python que también sirve de referencia para validar el firmware.

La Sección II resume los fundamentos teóricos. La Sección III describe la arquitectura, el enlace entre nodos y el diseño de ambos filtros. La Sección IV presenta y analiza los resultados, y las Secciones V y VI cierran con conclusiones y recomendaciones.

II. Marco teórico

Convolución y respuesta en frecuencia. Un sistema lineal e invariante en el tiempo responde con $y[n] = x[n] * h[n]$, que en frecuencia equivale a $Y(e^{j\omega}) = X(e^{j\omega})H(e^{j\omega})$ [2]. La respuesta $H(e^{j\omega}) = |H(e^{j\omega})|e^{j\angle H(e^{j\omega})}$ es compleja: la magnitud fija la atenuación de cada frecuencia y la fase su retardo.

FIR, IIR y transformada Z. La ecuación de diferencias

$$y[n] = \sum_{k=0}^{M} b_k x[n-k] - \sum_{k=1}^{N} a_k y[n-k] \tag{1}$$

tiene función de transferencia $H(z) = \sum b_k z^{-k}/(1 + \sum a_k z^{-k})$. Un sistema causal es estable si todos sus polos están dentro del círculo unitario, con región de convergencia $|z| > \max_k |p_k|$. Un FIR ($a_k = 0$) solo tiene polos en $z = 0$, es siempre estable y, con coeficientes simétricos, tiene fase lineal y retardo constante de $M/2$ muestras [3].

DFT, FFT y ventanas. La DFT de N muestras, $X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}$, cuesta $O(N^2)$; la FFT radix-2 la calcula en $O(N \log_2 N)$ si $N = 2^m$ [4]. La resolución es $\Delta f = f_s/N$. Truncar un bloque produce fuga espectral; la ventana Hann reduce los lóbulos laterales a cambio de un lóbulo principal más ancho [5]. Al usar Hann simétrica en análisis y síntesis, la superposición se normaliza por la suma de sus cuadrados para reconstruir [6]. Para que la IFFT de un espectro modificado sea real se debe conservar $X[N-k] = X^*[k]$.

Métricas. Con referencia limpia $s[n]$ y salida alineada $\hat{s}[n]$ sobre L muestras:

$$\mathrm{MSE} = \tfrac{1}{L}\sum (s-\hat{s})^2, \quad \mathrm{SNR}_{\mathrm{dB}} = 10\log_{10}\frac{\sum s^2}{\sum (s-\hat{s})^2}, \tag{2}$$

y $E_x = \sum |x[n]|^2$. La mejora es $\Delta\mathrm{SNR} = \mathrm{SNR}_{out} - \mathrm{SNR}_{in}$. La energía útil conservada es $100\, E_{\tilde{s}}/E_s$, con $\tilde{s}$ la referencia procesada por el mismo filtro, y la atenuación compara la energía del ruido de entrada con la del residuo $\hat{s} - \tilde{s}$.

III. Diseño e implementación

III-A. Arquitectura y flujo de datos

La Fig. 1 (Anexo A) muestra el sistema. El nodo 1 (RP2040) muestrea la entrada por su ADC de 12 bits a $f_s = 16$ kHz con DMA, en bloques de $N = 256$ (16 ms, $\Delta f = 62.5$ Hz) [7]. Elimina la media de cada bloque y, sobre una copia, aplica Hann y FFT para estimar el nivel, la frecuencia dominante y la saturación. Las muestras sin componente DC se envían a la ESP32, donde dos rutas con estado independiente ejecutan el FIR (nodo 2) y el método espectral (nodo 3), permitido por el enunciado al ser un microcontrolador de doble núcleo [1], [8]. La computadora solo almacena, ajusta parámetros y solicita la reproducción; no filtra. La terminal de la PC funciona como interfaz local del nodo 3: muestra el método activo, la energía conservada, el MSE y la SNR antes y después que calcula la ESP32.

Enlace entre nodos. Cada bloque viaja por SPI a 2 MHz en una trama de 1032 bytes: cabecera (2), identificador de secuencia (4), 256 muestras en float32 (1024), CRC-8 con polinomio $x^8 + x^2 + x + 1$ (1) y un byte de relleno [9]. La ESP32, esclava SPI [10], controla el flujo con una línea READY y rechaza tramas con longitud, cabecera o CRC inválidos o con muestras no finitas; el identificador revela saltos, duplicados y retrocesos. Cualquier pérdida o cola llena aborta la captura y queda registrada. El enlace con la PC usa paquetes PCM16 con índice y CRC-32 a 1 Mbaud, tras fallar el diagnóstico del puente USB a una tasa superior.

Justificación conjunta. Los tonos y bandas generados no superan 4 kHz, por lo que $f_s = 16$ kHz deja Nyquist en 8 kHz con margen [2]. El flujo útil es $16\,000 \times 4$ B $= 64$ kB/s, holgado frente a los 250 kB/s del SPI a 2 MHz. Hacia la PC viajan tres señales PCM16 (96 kB/s) sobre unos 99 kB/s útiles a 1 Mbaud: una ocupación cercana al 97 %, verificada con una captura completa sin pérdidas. Los bloques de 256 (FIR) y 1024 con salto de 512 (espectral) son potencias de dos [1]. Los estímulos se sintetizan a 16 kHz, por lo que su espectro queda acotado por construcción bajo Nyquist y las pruebas no presentan aliasing; una componente externa sobre 8 kHz sí se plegaría, pues el Sallen-Key filtra la salida del DAC, no la entrada del ADC [2].

III-B. Filtro temporal (nodo 2)

Con banda de paso hasta 1.6 kHz, rechazo desde 2.4 kHz, rizado de 0.1 dB y atenuación mínima de 60 dB se compararon un FIR por ventana Kaiser [11] y tres IIR (Tabla I). Se eligió el FIR de orden 74: es incondicionalmente estable (sus 74 polos están en $z = 0$, ROC $|z| > 0$), tiene fase lineal con retardo constante de 37 muestras (2.3 ms) y obtuvo la mayor SNR. Los IIR requieren menos operaciones, pero su retardo de grupo varía en la banda útil y deforma los transitorios. Su diagrama de polos y ceros y su respuesta en frecuencia están en la Fig. 3 (Anexo A).

Tabla I
Candidatos de filtro temporal (simulación, caso 2)

Filtro	Orden	MAC	Retardo	SNR (dB)
FIR Kaiser	74	75	37	65.1
Butterworth	20	50	18.4–24.7	15.5
Chebyshev I	10	25	13.3–20.6	14.8
Elíptico	6	15	6.0–10.6	18.6

En la ESP32 el FIR procesa muestra a muestra con un anillo de 75 posiciones en float32 y coeficientes exportados desde Python, con $H(z) = \sum_{k=0}^{74} b_k z^{-k}$. La salida se convierte a PCM16 con saturación explícita y, para la reproducción, a 8 bits del DAC, lo que limita el rango dinámico a unos 48 dB. En los WAV de la ESP32, un ajuste conjunto de senos y cosenos sobre 93 bloques, excluyendo el arranque, difiere menos de 0.0035 dB de la respuesta calculada en 440/880/1320 Hz. A 4 kHz solo se resuelve una cota de -58.4 dB, frente a -73.0 dB calculados: la cuantización PCM16 limita esta medida digital antes del DAC.

III-C. Filtro espectral y evaluación (nodo 3)

El método procesa bloques de $N = 1024$ ($\Delta f = 15.625$ Hz) con salto de 512:

1. multiplicar el bloque por Hann;
2. calcular la FFT radix-2 [4];
3. proteger los bins bajo f_{min} (1.6 kHz);
4. marcar como ruido cada bin no protegido cuya energía $|X[k]|^2$ supere un umbral relativo (0.05) a la máxima energía de esa región, con piso de 0.001 del pico global, junto con sus vecinos dentro de un ancho total de 100 Hz;
5. multiplicar esos bins y sus conjugados por un factor real (0.1);
6. calcular la IFFT, volver a aplicar Hann y reconstruir por superposición y suma normalizada por $\sum w^2$ [6].

El umbral se recalcula en cada bloque, por lo que la selección sigue la interferencia si cambia de frecuencia o de nivel [12]. Los cuatro parámetros se cambian en ejecución desde la terminal, sin recompilar. El mismo nodo acumula, en doble precisión, MSE, SNR de entrada y salida, energía útil conservada, atenuación de la interferencia y tiempo por bloque.

III-D. Reconstrucción analógica

La salida del DAC pasa por un divisor de 10 kΩ/10 kΩ, un seguidor (LM358, amplificador A) y un Sallen-Key pasabajos de ganancia unitaria (amplificador B) con $R_1 = R_2 = 4.7$ kΩ, $C_1 = 22$ nF de realimentación y $C_2 = 10$ nF a tierra [13]–[15], alimentado a 5 V. Su frecuencia de corte $f_c = 1/(2\pi\sqrt{R_1 R_2 C_1 C_2}) \approx 2283$ Hz y su $Q = \frac{1}{2}\sqrt{C_1/C_2} \approx 0.74$ atenúan las imágenes del DAC antes del amplificador. Se usó 22 nF, valor comercial disponible, en lugar de los 20 nF de un Butterworth exacto ($Q = 0.707$, 2394 Hz); el cambio produce un ligero realce de $+0.3$ % bajo 1 kHz. Con el sistema en marcha se midieron 5 V en la alimentación y 0.815 V en la

entrada y la salida del seguidor, la mitad del nivel medio del DAC, como predice el divisor.

Para comparar las amplitudes muestreadas se usa $|H_{med}(f)| = |H_{SK}(f)|\,|\mathrm{sinc}(f/16000)|$: la retención del DAC atenúa la fundamental [16]. La curva representa esa componente, sin ajustar los componentes físicos ni modelar imágenes que se pliegan al adquirir a 16 kHz.

IV. Resultados y análisis

Criterios de aceptación, fijados antes de las pruebas finales: con ruido fuera de banda, $\Delta\mathrm{SNR} \geq 10$ dB y energía útil conservada ≥ 90 %; en todos los casos, tiempo máximo por bloque menor que su duración (16 ms FIR, 32 ms espectral); cero pérdidas de bloque. Para el filtro analógico, error de ganancia ≤ 15 % por tono con variabilidad ≤ 10 %. La concordancia entre la salida de la placa y el modelo de Python, $100\,\mathrm{máx}(0, 1 - \mathrm{RMSE}/\mathrm{RMS})$, debe ser ≥ 90 % para ambas rutas: admite NRMSE hasta 0.1 y comprueba equivalencia, no eliminación de ruido. El criterio analógico se fijó antes de medir; la corrección física de retención se documentó después como reanálisis, conservando las medidas y los criterios originales.

Equivalencia modelo–firmware. Los mismos casos se evaluaron en Python (float64) [17], [18] y en la ESP32 (float32) sobre 95 963 muestras, tras compensar las 37 muestras del FIR. La sesión controlada del 5/11 procesó el CSV histórico multitono con interferencia tonal (P2 tonal), sin pasar por el ADC: SNR espectral 10.0017 → 29.9764 dB en placa. Sobre el mismo PCM16, Python da 29.97643172 dB, con diferencia 8.2×10^{-8} dB; la concordancia es 100 % FIR y 99.99966 % espectral. El valor anterior 29.9760 dB provenía del modelo sin esa cuantización de entrada. Una captura física distinta de 6 s con micrófono entregó 375 bloques, 96 000 muestras a 16 000.26 muestras/s, sin pérdidas, errores SPI detectados ni recortes. Su máximo conjunto fue 16.323 ms frente al salto de 32 ms; sin referencia limpia no acredita SNR. Las tablas y figuras seleccionadas se adjuntan al proyecto; los registros completos son locales.

Tabla II
Resultados digitales ESP32, 95 963 muestras a 16 kHz. Banco: 160 MHz, 30/10; PC: CSV en placa, 240 MHz, 5/11.

Caso	Método	$\mathrm{SNR}_{\mathrm{in}}$ (dB)	ΔSNR (dB)	MSE	Útil (%)	t_{max} (ms)
P2 tonal: banco	FIR	10.0	59.05	1.7×10^{-8}	100.0	8.83
	Espectral	10.0	19.97	1.3×10^{-4}	100.0	18.86
P2: banco	FIR	5.0	60.14	1.5×10^{-8}	100.0	8.83
	Espectral	5.0	18.35	2.3×10^{-4}	100.0	18.95
P5: banco	FIR	0.0	0.003	2.2×10^{-2}	100.0	8.80
	Espectral	0.0	0.002	2.2×10^{-2}	100.0	18.84
P2 tonal: PC	FIR	10.0017	59.0199	1.67×10^{-8}	100.0	–
	Espectral	10.0017	19.9747	1.34×10^{-4}	100.0	–

Comparación de métodos. Con ruido fuera de banda (P2 tonal y P2) el FIR mejora la SNR unos 60 dB porque atenúa al menos 60 dB toda la banda de rechazo, mientras el espectral queda limitado a unos 20 dB por su factor de 0.1. Ambos conservan prácticamente toda la energía útil. Los tiempos máximos (8.8 ms y 18.9 ms) cumplen sus presupuestos con margen a 160 MHz; la versión final corre a 240 MHz con el procesamiento en un núcleo distinto de las interrupciones. No se repitió ese banco completo: la evaluación PC tiene máximo conjunto de 25.694 ms y estado DSP de 80 960 bytes, frente a los 53 272 del banco anterior. Esas medidas preceden a la instrumentación por ruta y regional nueva; no son tiempos individuales ni memoria de esa versión. La latencia algorítmica del FIR es su retardo de 2.3 ms más el bloque de 16 ms; la del espectral es del orden de una ventana completa, 64 ms, más 2.3 ms de retardo añadido para alinearla con el FIR.

Compromiso ruido–distorsión. En P5 el ruido comparte la banda útil y ninguno de los dos métodos mejora la SNR, porque proteger la señal útil también protege el ruido superpuesto. Al bajar f_{min} a 600 Hz, el espectral mejora 5.9 dB con factor 0.1 pero conserva solo 72 % de la energía útil, frente a 1.8 dB y 90 % con factor 0.8 (Fig. 2, Anexo A).

Comparación analógica. La captura del 5/11 a las 18:57 UTC cumple en los tres tonos útiles (Tabla III). A 4 kHz la amplitud varía 19.3 %, por encima del 10 % fijado: ese punto no permite validar el criterio. La imagen del DAC en 12 kHz se pliega sobre 4 kHz al adquirir a 16 kHz; el modelo nominal estima una contribución cercana al 4 % de la fundamental filtrada. Es una limitación posible, sin atribuirle todo el error observado ni relajar los criterios.

Tabla III
Ganancia analógica medida frente a Sallen-Key con retención DAC.

Hz	Modelo	Medida	Error (%)	Estado
440	0.5007	0.4881	2.52	Cumple
880	0.4987	0.4958	0.59	Cumple
1320	0.4823	0.4343	9.95	Cumple
4000	0.1433	0.0972	–	Inválida

Protocolo de prueba. Cada caso se ejecuta de dos formas. Para las métricas, la PC envía a la ESP32 el CSV con la mezcla y la referencia $s[n]$ generadas por el modelo; la ESP32 filtra y calcula en placa MSE, SNR, ΔSNR, energía útil, atenuación y tiempos. Para la cadena física, el audio del caso suena frente al micrófono y recorre Pico, SPI y ESP32; sin referencia alineada no hay métricas de calidad: se verifican tiempos, pérdidas y espectros, y la salida se juzga escuchándola. P1 es un tono con interferencia de 4 kHz; P3, un barrido con ruido blanco; P4, transitorios con impulsos; P2 y P5 conservan los CSV históricos. Faltan las tres repeticiones de P1–P5; el retardo analógico no se midió y P6 se ejecuta en defensa. *Pendiente: tabla de P1–P5 con tres repeticiones por condición, parámetros, media y variabilidad; distinguir evaluación CSV en placa y adquisición ADC.*

V. Conclusiones

La arquitectura implementada mantiene la adquisición separada en la Pico y la ESP32 ejecuta dos rutas de filtrado

independientes dentro de su presupuesto de tiempo, con un enlace que delimita bloques, detecta errores y reporta pérdidas; su verificación integral física sigue pendiente. La coincidencia entre el modelo de Python y el firmware muestra que el diseño validado en simulación se trasladó al microcontrolador sin cambiar su comportamiento. Cuando el ruido está separado en frecuencia, el FIR ofrece la mayor reducción con fase lineal y menor latencia, mientras que el método espectral reduce menos pero se adapta por bloque a interferencias cambiantes sin rediseñar coeficientes. Cuando ruido y señal comparten frecuencias, ningún filtro lineal ni ganancia espectral puede separarlos sin quitar información útil, y la elección de parámetros pasa a ser una decisión de compromiso entre ruido residual y distorsión.

VI. Recomendaciones

Se recomienda colocar un filtro antialias analógico antes del ADC de la Pico y medir su respuesta, ya que la protección contra aliasing hoy depende del contenido de la señal. Un DAC externo de mayor resolución o una salida PWM de alta frecuencia mejoraría el rango dinámico de la reproducción, limitado por los 8 bits del DAC interno. En el método espectral convendría estimar el piso de ruido durante silencios y aplicar sustracción espectral con un suelo mínimo de ganancia, para mejorar el caso de ruido superpuesto sin introducir ruido musical. Finalmente, ejecutar cada ruta en un núcleo distinto de la ESP32 permitiría reducir la latencia de extremo a extremo.

Referencias

[1] Instituto Tecnológico de Costa Rica, "Proyecto grupal 1: Diseño de un sistema distribuido de reducción adaptativa de ruido en tiempo real," CE 1110 Análisis de Señales Mixtas, Cartago, Costa Rica, 2026.

[2] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[3] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2007.

[4] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, 1965.

[5] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, 1978.

[6] J. B. Allen, "Short term spectral analysis, synthesis, and modification by discrete Fourier transform," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 25, no. 3, pp. 235–238, 1977.

[7] Raspberry Pi Ltd, *RP2040 Datasheet: A microcontroller by Raspberry Pi*, Raspberry Pi Ltd, 2024, sec. 4.9, ADC. [Online]. Available: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf.

[8] Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems, 2025, [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

[9] W. W. Peterson and D. T. Brown, "Cyclic codes for error detection," *Proc. IRE*, vol. 49, no. 1, pp. 228–235, 1961.

[10] Espressif Systems, *ESP-IDF Programming Guide v5.5: DAC, SPI Slave Driver and ESP Timer*, Espressif Systems, 2025, [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/.

[11] J. F. Kaiser, "Nonrecursive digital filter design using the I_0-sinh window function," in *Proc. IEEE Int. Symp. Circuits Syst.*, 1974, pp. 20–23.

[12] S. F. Boll, "Suppression of acoustic noise in speech using spectral subtraction," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 27, no. 2, pp. 113–120, 1979.

[13] R. P. Sallen and E. L. Key, "A practical method of designing RC active filters," *IRE Trans. Circuit Theory*, vol. 2, no. 1, pp. 74–85, 1955.

[14] J. Karki, "Analysis of the Sallen-Key architecture," Texas Instruments, Tech. Rep. SLOA024B, 2002.

[15] Texas Instruments, *LM358, LM258, LM2904: Industry-Standard Dual Operational Amplifiers*, Texas Instruments, 2024, sLOS068AB, rev. Oct. 2024.

[16] ——, *DAC Output Response*, precision Labs TIPL4705. [Online]. Available: https://www.ti.com/content/dam/videos/external-videos/en-us/2/3816841626001/5577272810001.mp4/subassets/TIPL4705-DAC-Output-Response.pdf.

[17] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[18] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

Apéndice A
Figuras complementarias

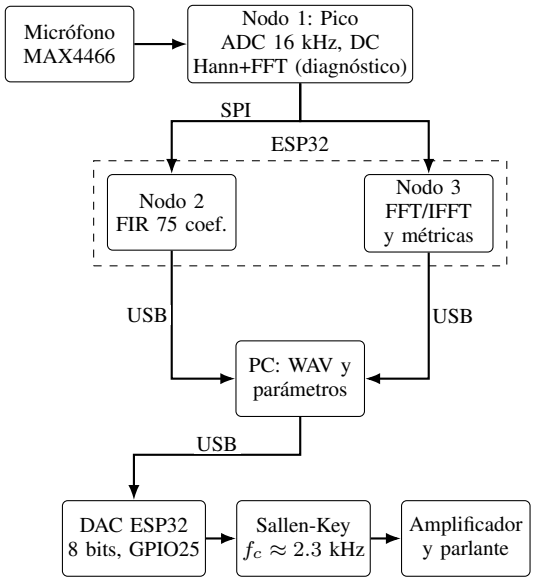


Figura 1. Arquitectura. Nodos 2 y 3 en la ESP32 con rutas independientes; ambas salidas comparten el USB y la PC devuelve la elegida al DAC.

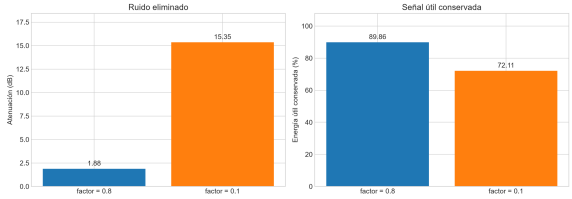


Figura 2. P5 con $f_{min} = 600$ Hz: más supresión implica menos energía útil conservada.

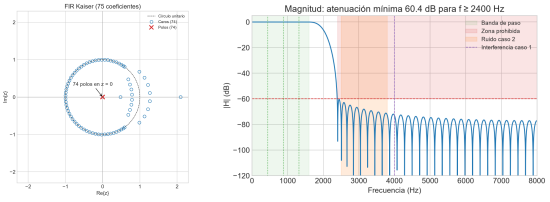


Figura 3. FIR Kaiser: polos y ceros; magnitud con rechazo $\geq$ 60 dB desde 2.4 kHz. Fase, retardo, IIR, FIR medido y respuesta analógica en las evidencias del repositorio.